

Nonlinear Dynamics: Selected Notebook Excerpts

University of Edinburgh
Chenrui Xu

Scope

Selected sections from a completed Honours Differential Equations course assignment. The work combines symbolic Jacobian and eigenvalue calculations with numerical integration, phase portraits, oscillator analysis and visualisations of the Rössler system.

What is included

Original PDF pages 1-17 and 22-27 are reproduced after this cover. Page 22 begins at Question 4; the preceding plot from Question 3 is omitted. Original page numbers and assignment prompts are retained. Student identification has been removed.

Selection note

This excerpt focuses on nonlinear systems and trajectories. A separate numerical-methods exercise on original pages 18-21 is omitted. The complete historical report is retained in the private archive.

Source note

Portfolio excerpt assembled in September 2026 from the 27-page report dated December 1, 2022. It is selected coursework using established mathematical models and numerical methods, not a claim of original algorithms or a peer-reviewed publication.

Project_assignment_template

December 1, 2022

1 Honours Differential Equations

1.1 Project Assignment

Due: Friday 2nd December 2022, noon

2 Chenrui Xu

3

3.1 Question 1

Please **clearly** indicate where you answer each sub question by using a markdown cell.

3.2 A

```
[1]: import sympy as sym
import numpy as np
sym.init_printing()
from IPython.display import display_latex

x = sym.Function('x')
y = sym.Function('y')
t = sym.Symbol("t")
a = sym.Symbol("a")
b = sym.Symbol("b")
u = sym.Function('u')
v = sym.Function('v')

equ1 = sym.Eq(x(t).diff(t), y(t))
equ2 = sym.Eq(y(t).diff(t), -x(t) + a*(y(t) - (y(t))**3/3))
[equ1, equ2]
```

[1]:
$$\left[\frac{d}{dt}x(t) = y(t), \frac{d}{dt}y(t) = a \left(-\frac{y^3(t)}{3} + y(t) \right) - x(t) \right]$$

```
[2]: FG = sym.Matrix([equ1.rhs, equ2.rhs])
matJ = FG.jacobian([x(t), y(t)])
matJ
```

[2]:
$$\begin{bmatrix} 0 & 1 \\ -1 & a(1 - y^2(t)) \end{bmatrix}$$

```
[3]: lin_mat = matJ.subs({x(t):0, y(t):0})
lin_mat * sym.Matrix([u(t),v(t)])
```

[3]:
$$\begin{bmatrix} v(t) \\ av(t) - u(t) \end{bmatrix}$$

3.3 Find eigenvalues

```
[6]: envals = list(lin_mat.eigenvals().keys())
envals
```

[6]:
$$\left[\frac{a}{2} - \frac{\sqrt{(a-2)(a+2)}}{2}, \frac{a}{2} + \frac{\sqrt{(a-2)(a+2)}}{2} \right]$$

The results show in the order of r_1, r_2 .

3.4 Behaviour of critical point with changing values

```
[7]: envals[0].subs({a:-2})
for i in range(-5,-2):
    print("When a is equal to : " + str(i))
    print("check if they are both less than 0")
    #check if they are both less than 0
    display_latex([envals[0].subs({a:i}) < 0 ,envals[1].subs({a:i}) < 0 ])
for i in range(-1,2):
    print("When a is equal to : " + str(i))
    print("complex eigenvalues")
    display_latex([envals[0].subs({a:i}) ,envals[1].subs({a:i})])
for i in range(2,5):
    print("When a is equal to : " + str(i))
    print("check if they are both less than 0")
    #check if they are both less than 0
    display_latex([envals[0].subs({a:i}) < 0 ,envals[1].subs({a:i}) < 0])
```

When a is equal to : -5
check if they are both less than 0

[True, True]

When a is equal to : -4
check if they are both less than 0

[True, True]

When a is equal to : -3
check if they are both less than 0

[True, True]

When a is equal to : -1
complex eigenvalues

$$\left[-\frac{1}{2} - \frac{\sqrt{3}i}{2}, -\frac{1}{2} + \frac{\sqrt{3}i}{2} \right]$$

When a is equal to : 0
complex eigenvalues

$$[-i, i]$$

When a is equal to : 1
complex eigenvalues

$$\left[\frac{1}{2} - \frac{\sqrt{3}i}{2}, \frac{1}{2} + \frac{\sqrt{3}i}{2} \right]$$

When a is equal to : 2
check if they are both less than 0

[False, False]

When a is equal to : 3
check if they are both less than 0

[False, False]

When a is equal to : 4
check if they are both less than 0

[False, False]

When $a \leq -2$,

$r_2 < r_1 < 0$ and they are real, so the critical point is a stable sink node.

When $-2 < a < 0$,

r_1, r_2 are complex and real parts are smaller than 0, so the critical point is an stable focus.

When $a = 0$,

r_1, r_2 are complex and only have imaginary parts, so the critical point is an stable center.

When $0 < a < 2$,

r_1, r_2 are complex and real parts are greater than 0, so the critical point is an unstable focus.

When $a \geq 2$,

$r_1 > r_2 > 0$ and they are real, so the critical point is an unstable source node.

3.5 B)

```
[6]: import numpy as np
from matplotlib import pyplot as plt
from scipy.integrate import odeint
%matplotlib inline

# Define vector field
def vField(x,t):
    u = x[1]
    v = -x[0]+.1*(x[1]-((x[1]**3)/3))
    return [u,v]

# Plot vector field
X, Y = np.mgrid[-10:10:40j,-10:10:30j]
U, V = vField([X,Y],0)

# Define colours for each vector based on their lengths
M = np.hypot(U, V)

fig, ax = plt.subplots(figsize=(10, 7))
ax.quiver(X, Y, U, V, M, scale=1/0.01, pivot = 'mid', cmap = plt.cm.bone)

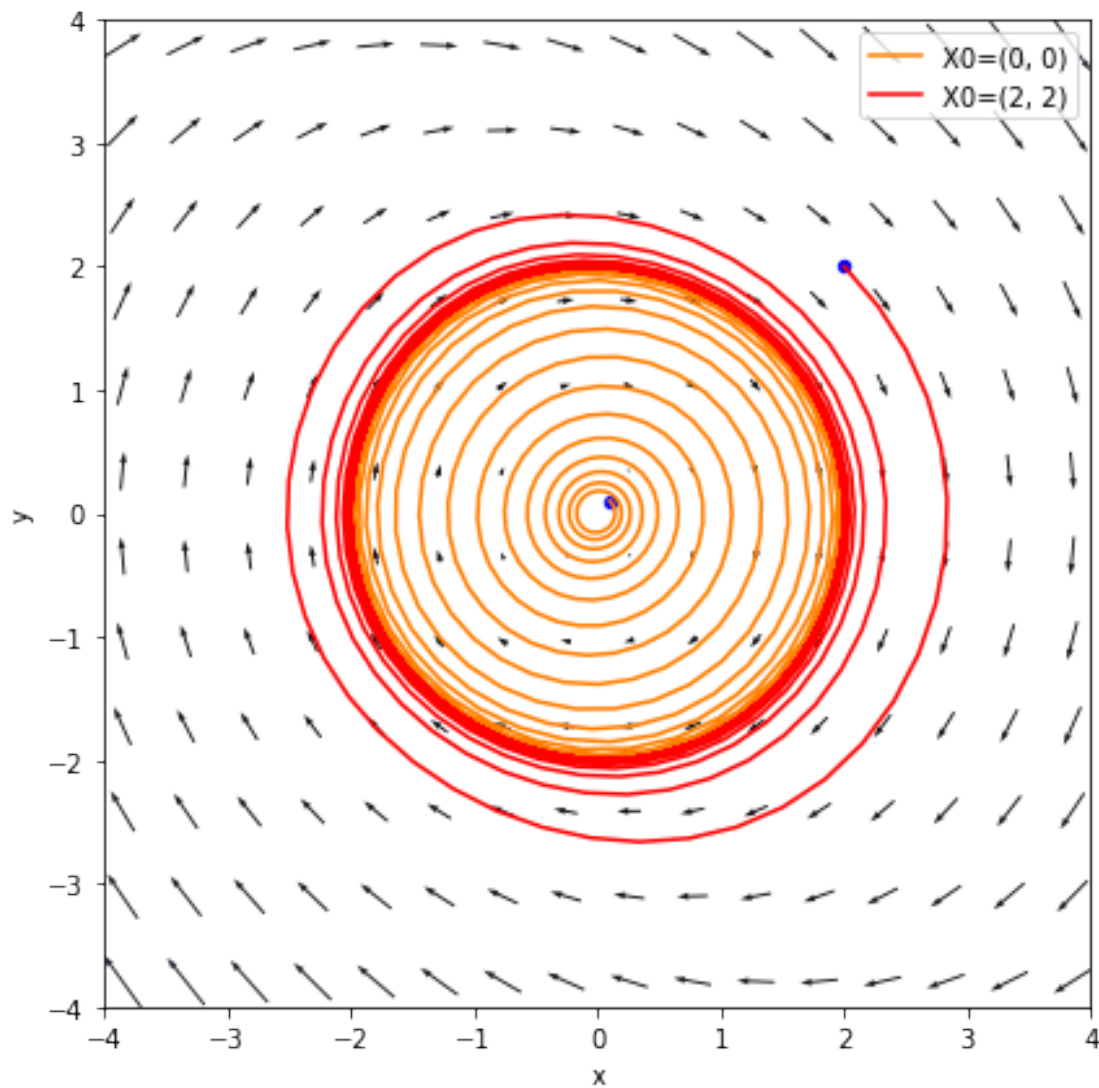
# Settings for trajectories
ics = [[0.1,0.1],[2,2]]
durations = [[0,200],[0,150]]
vcolors = plt.cm.autumn_r(np.linspace(0.5, 1., len(ics))) # colors for each
↳trajectory

# Plot trajectories
for i, ic in enumerate(ics):
    t = np.linspace(durations[i][0], durations[i][1],1000)
    x = odeint(vField, ic, t)
    ax.plot(x[:,0], x[:,1], color=vcolors[i], label='X0=(%.f, %.f)' % (ic[0],
↳ic[1]) )

ic_x = [ic[0] for ic in ics]
ic_y = [ic[1] for ic in ics]
ax.scatter(ic_x, ic_y, color='blue', s=20)

plt.xlabel('x')
plt.ylabel('y')
plt.xlim(-4,4)
plt.ylim(-4,4)
plt.legend()
plt.gca().set_aspect(1.0)
```

```
plt.show()
```



```
[7]: import numpy as np
from matplotlib import pyplot as plt
from scipy.integrate import odeint
%matplotlib inline

# Define vector field
def vField(x,t):
    u = x[1]
    v = -1*x[0]-0.1*(x[1]-(x[1]**3)/3)
    return [u,v]
```

```

# Plot vector field
X, Y = np.mgrid[-15:15:30j,-15:15:30j]
U, V = vField([X,Y],0)

# define colours for each vector based on their lengths
M = np.hypot(U, V)

fig, ax = plt.subplots(figsize=(10, 7))
ax.quiver(X, Y, U, V, M, scale=1/0.01, pivot = 'mid', cmap = plt.cm.bone)

# Settings for trajectories
ics = [[0.1,0.1],[2,2]]
durations = [[0,200],[0,6]]
vcolors = plt.cm.autumn_r(np.linspace(0.5, 1., len(ics))) # colors for each
↳trajectory

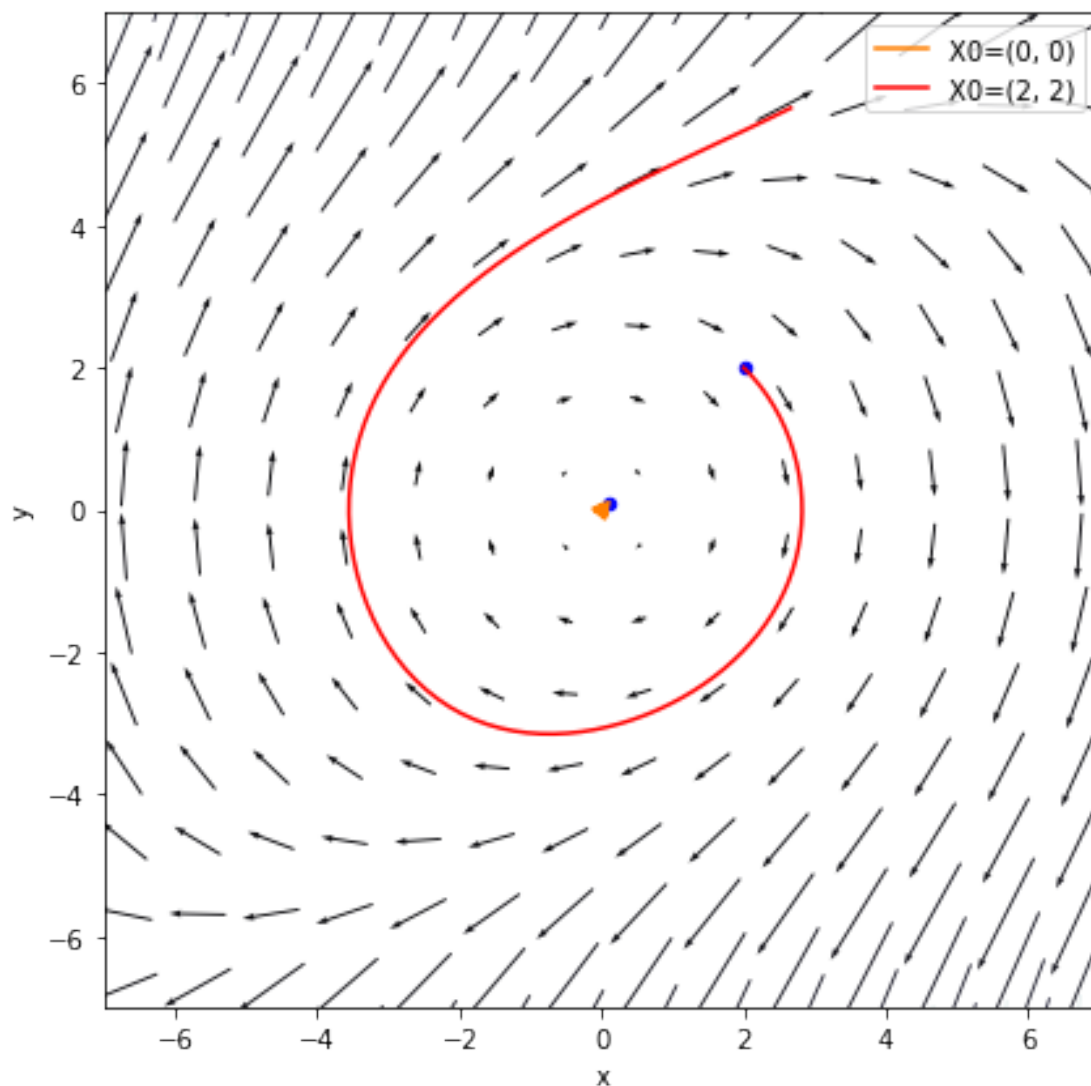
# plot trajectories
for i, ic in enumerate(ics):
    t = np.linspace(durations[i][0], durations[i][1],100)
    x = odeint(vField, ic, t)
    ax.plot(x[:,0], x[:,1], color=vcolors[i], label='X0=(%.f, %.f)' % (ic[0],
↳ic[1]))

ic_x = [ic[0] for ic in ics]
ic_y = [ic[1] for ic in ics]
ax.scatter(ic_x, ic_y, color='blue', s=20)

plt.xlabel('x')
plt.ylabel('y')
plt.xlim(-7,7)
plt.ylim(-7,7)
plt.legend()
plt.gca().set_aspect(1.0)

plt.show()

```



3.6 Question 2

3.7 (A)

```
[9]: x = sym.Function('x')
y = sym.Function('y')
t = sym.Symbol("t")
a = sym.Symbol("a")
b = sym.Symbol("b")
u = sym.Function('u')
v = sym.Function('v')

equ1 = sym.Eq(x(t).diff(t), a - x(t) - b*x(t) + x(t)**2*y(t))
```



```
equ2 = sym.Eq(y(t).diff(t), b*x(t)-x(t)**2*y(t))
[equ1, equ2]
```

[9]: $\left[\frac{d}{dt}x(t) = a - bx(t) + x^2(t)y(t) - x(t), \frac{d}{dt}y(t) = bx(t) - x^2(t)y(t) \right]$

```
[10]: FG = sym.Matrix([equ1.rhs, equ2.rhs])

print("a = " + str(sym.solve(FG)[0].get(a)))
print("b = " + str(sym.solve(FG)[0].get(b)))
print("x(t) = a and y(t) = b/a")
sym.solve(FG)
```

```
a = x(t)
b = x(t)*y(t)
x(t) = a and y(t) = b/a
```

[10]: $\{a : x(t), b : x(t)y(t)\}$

```
[11]: FG = sym.Matrix([equ1.rhs, equ2.rhs])
matJ = FG.jacobian([x(t), y(t)])
display_latex(matJ)
lin_mat = matJ.subs({x(t):a, y(t):(b/a)})
print("when we sub in x(t) = a and y(t) = b/a")
display_latex(lin_mat)
```

$$\begin{bmatrix} -b + 2x(t)y(t) - 1 & x^2(t) \\ b - 2x(t)y(t) & -x^2(t) \end{bmatrix}$$

when we sub in $x(t) = a$ and $y(t) = b/a$

$$\begin{bmatrix} b - 1 & a^2 \\ -b & -a^2 \end{bmatrix}$$

3.8 Find eigenvalues and eigenvectors

```
[12]: lin_mat.eigenvects()
```

[12]: $\left[\left(-\frac{a^2}{2} + \frac{b}{2} - \frac{\sqrt{(a^2 - 2a - b + 1)(a^2 + 2a - b + 1)}}{2} - \frac{1}{2}, 1, \left[\left[-\frac{a^2}{b} - \frac{-\frac{a^2}{2} + \frac{b}{2} - \frac{\sqrt{(a^2 - 2a - b + 1)(a^2 + 2a - b + 1)}}{2}}{1} - \frac{1}{2} \right] \right] \right), \left(-\frac{a^2}{2} + \frac{b}{2} + \frac{\sqrt{(a^2 - 2a - b + 1)(a^2 + 2a - b + 1)}}{2} - \frac{1}{2}, 1, \left[\left[-\frac{a^2}{b} - \frac{-\frac{a^2}{2} + \frac{b}{2} + \frac{\sqrt{(a^2 - 2a - b + 1)(a^2 + 2a - b + 1)}}{2}}{1} - \frac{1}{2} \right] \right] \right)$

3.9 The results show in the order of eigenvalues, algebraic multiplicity, eigenvector.

3.10 (B)

```
[13]: import numpy as np
from matplotlib import pyplot as plt
from scipy.integrate import odeint
%matplotlib inline
```

```

# Define vector field
def vField(x,t):
    u = 1 - x[0] - 0.5*x[0] + (x[0]**2)*x[1]
    v = 0.5*x[0] - x[0]**2*x[1]
    return [u,v]

# Plot vector field
X, Y = np.mgrid[0:5:20j,0:5:20j]
U, V = vField([X,Y],0)

# Define colours for each vector based on their lengths
M = np.hypot(U, V)

fig, ax = plt.subplots(figsize=(10, 7))
ax.quiver(X, Y, U, V, M, scale=1/0.001, pivot = 'mid', cmap = plt.cm.bone)

# Settings for trajectories
ics = [[0,0],[2,3]]
durations = [[0,10],[0,10]]
vcolors = plt.cm.autumn_r(np.linspace(0.5, 1., len(ics)))
# colors for each trajectory

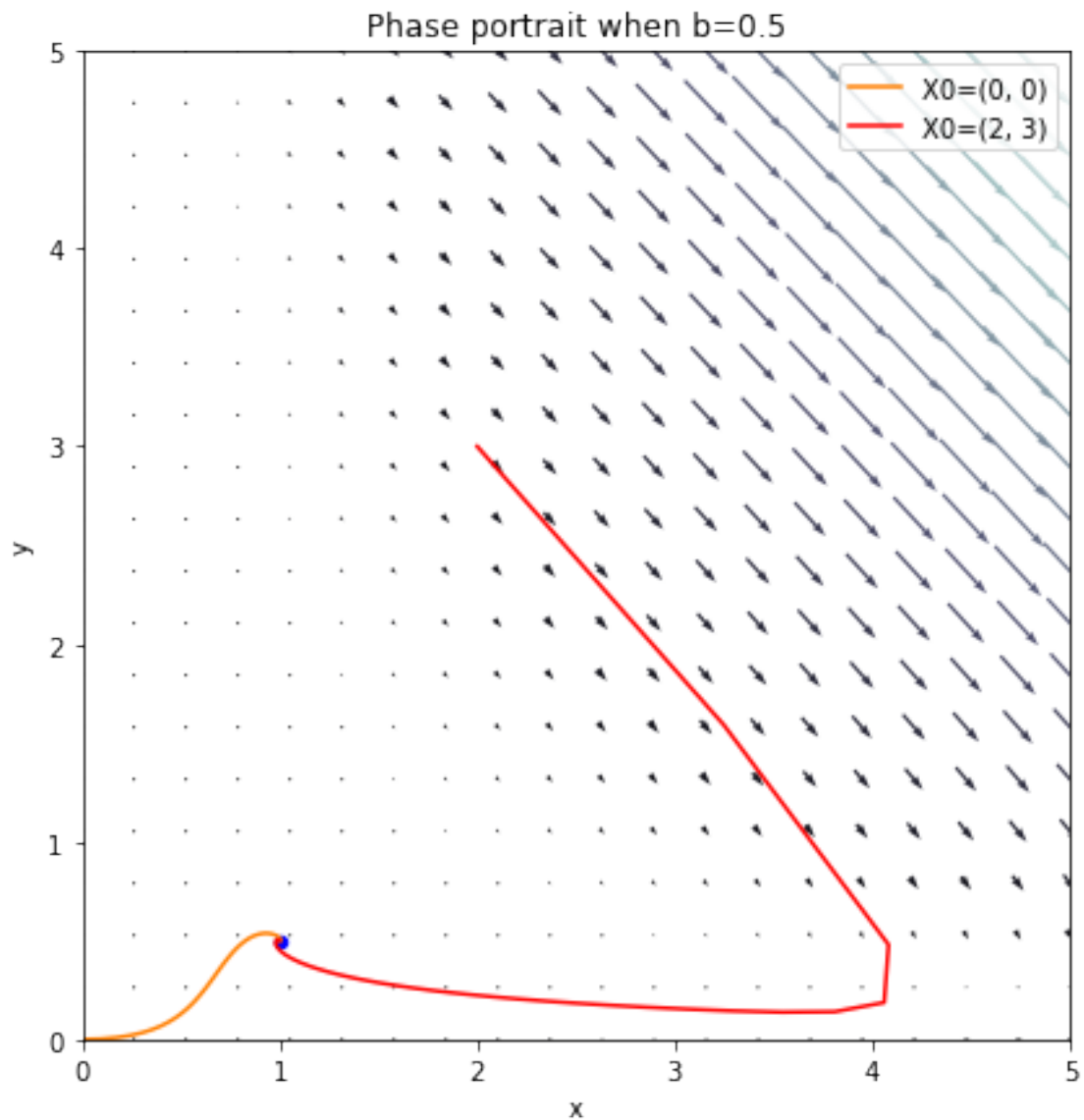
# Plot trajectories
for i, ic in enumerate(ics):
    t = np.linspace(durations[i][0], durations[i][1],100)
    x = odeint(vField, ic, t)
    ax.plot(x[:,0], x[:,1], color=vcolors[i], label='X0=(%.f, %.f)' % (ic[0],
↵ic[1]))

# Reveal the critical point
ax.scatter(1, 0.5, color='blue', s=20)

plt.xlabel('x')
plt.ylabel('y')
plt.xlim(0,5)
plt.ylim(0,5)
plt.legend()
plt.gca().set_aspect(1.0)
plt.title('Phase portrait when b=0.5')

plt.show()

```



```
[14]: import numpy as np
from matplotlib import pyplot as plt
from scipy.integrate import odeint
%matplotlib inline

# Define vector field
def vField(x,t):
    u = 1- x[0] - 3*x[0] + x[0]**2*x[1]
    v = 3*x[0] - x[0]**2*x[1]
    return [u,v]

# Plot vector field
```

```

X, Y = np.mgrid[0:5:20j,0:5:20j]
U, V = vField([X,Y],0)

# define colours for each vector based on their lengths
M = np.hypot(U, V)

fig, ax = plt.subplots(figsize=(10, 7))
ax.quiver(X, Y, U, V, M, scale=1/0.001, pivot = 'mid', cmap = plt.cm.bone)

# Settings for trajectories
ics = [[0,0],[2,3]]
durations = [[0,10],[0,10]]
vcolors = plt.cm.autumn_r(np.linspace(0.5, 1., len(ics))) # colors for each
↳ trajectory

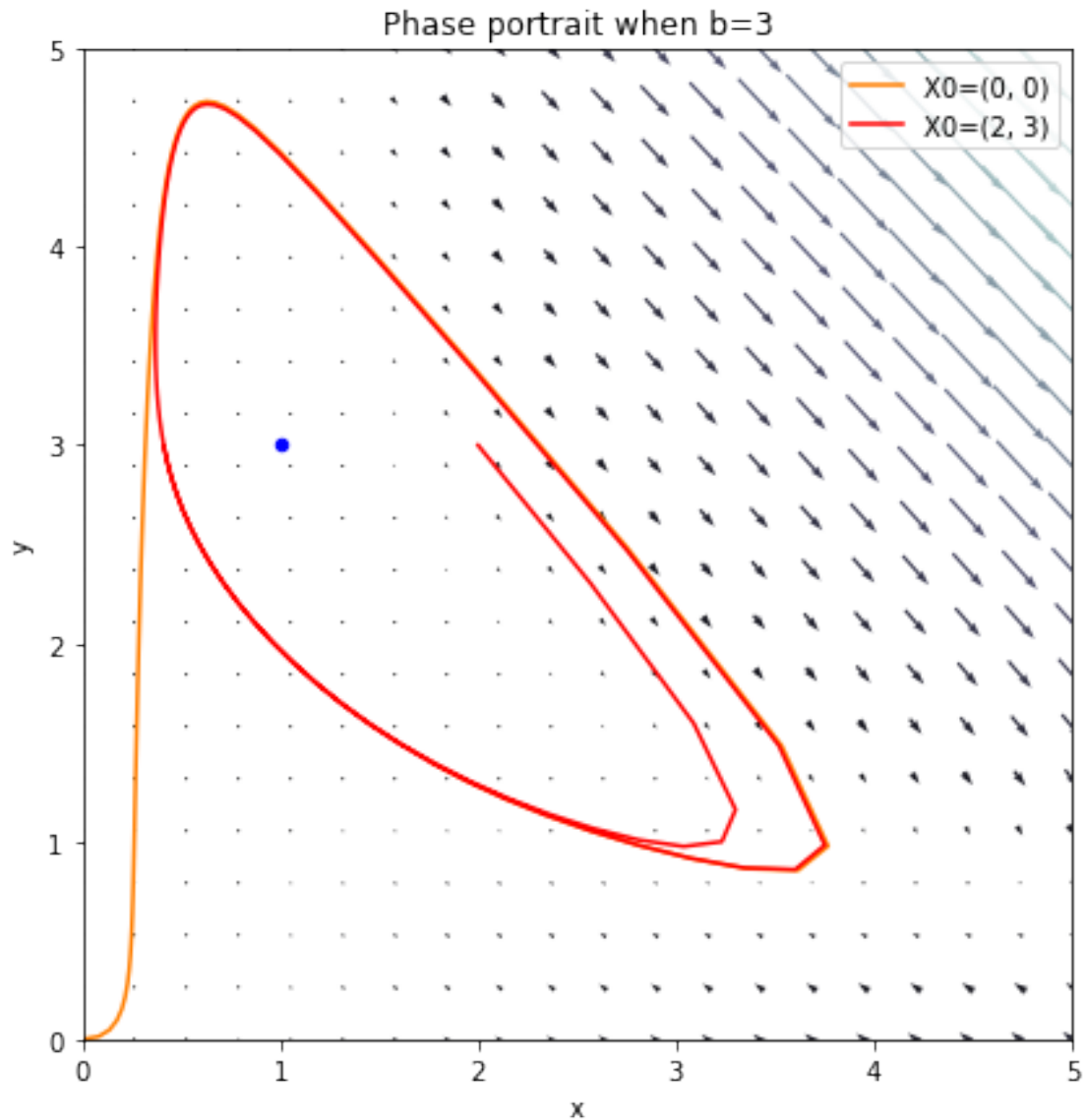
# plot trajectories
for i, ic in enumerate(ics):
    t = np.linspace(durations[i][0], durations[i][1],100)
    x = odeint(vField, ic, t)
    ax.plot(x[:,0], x[:,1], color=vcolors[i], label='X0=(%.f, %.f)' % (ic[0],
↳ ic[1]) )

# Reveal the critical point
ax.scatter(1, 3, color='blue', s=20)

plt.xlabel('x')
plt.ylabel('y')
plt.xlim(0,5)
plt.ylim(0,5)
plt.legend()
plt.gca().set_aspect(1.0)
plt.title('Phase portrait when b=3')

plt.show()

```



According to plots above, we could conclude that:

When $b = 0.5$, the phase portrait has solutions spiral into the critical point $(1, 0.5)$, which is a stable spiral sink. When $b = 3$, the phase portrait has solutions spiral away from the critical point $(1, 3)$ and asymptote to a limit cycle, which is an unstable spiral source.

3.11 (C)

```
[18]: import numpy as np
from matplotlib import pyplot as plt
from scipy.integrate import odeint
%matplotlib inline
```

```

bs = [1.4, 2.4]
for b in bs:

    # Define vector field
    def vField(x,t):
        u = 1 - x[0] - b*x[0] + x[0]**2*x[1]
        v = b*x[0] - x[0]**2*x[1]
        return [u,v]

    # Plot vector field
    X, Y = np.mgrid[0:5:20j, 0:5:20j]
    U, V = vField([X,Y], 0)

    # define colours for each vector based on their lengths
    M = np.hypot(U, V)

    fig, ax = plt.subplots(figsize=(10, 7))
    ax.quiver(X, Y, U, V, M, scale=1/0.001, pivot = 'mid', cmap = plt.cm.bone)

    # Settings for trajectories
    ics = [[0,0],[2,3]]
    durations = [[0,15],[0,15]]
    vcolors = plt.cm.autumn_r(np.linspace(0.5, 1., len(ics))) # colors for
    ↪ each trajectory

    # plot trajectories
    for i, ic in enumerate(ics):
        t = np.linspace(durations[i][0], durations[i][1], 100)
        x = odeint(vField, ic, t)
        ax.plot(x[:,0], x[:,1], color=vcolors[i], label='X0=(%.f, %.f)' %
    ↪ (ic[0], ic[1]))

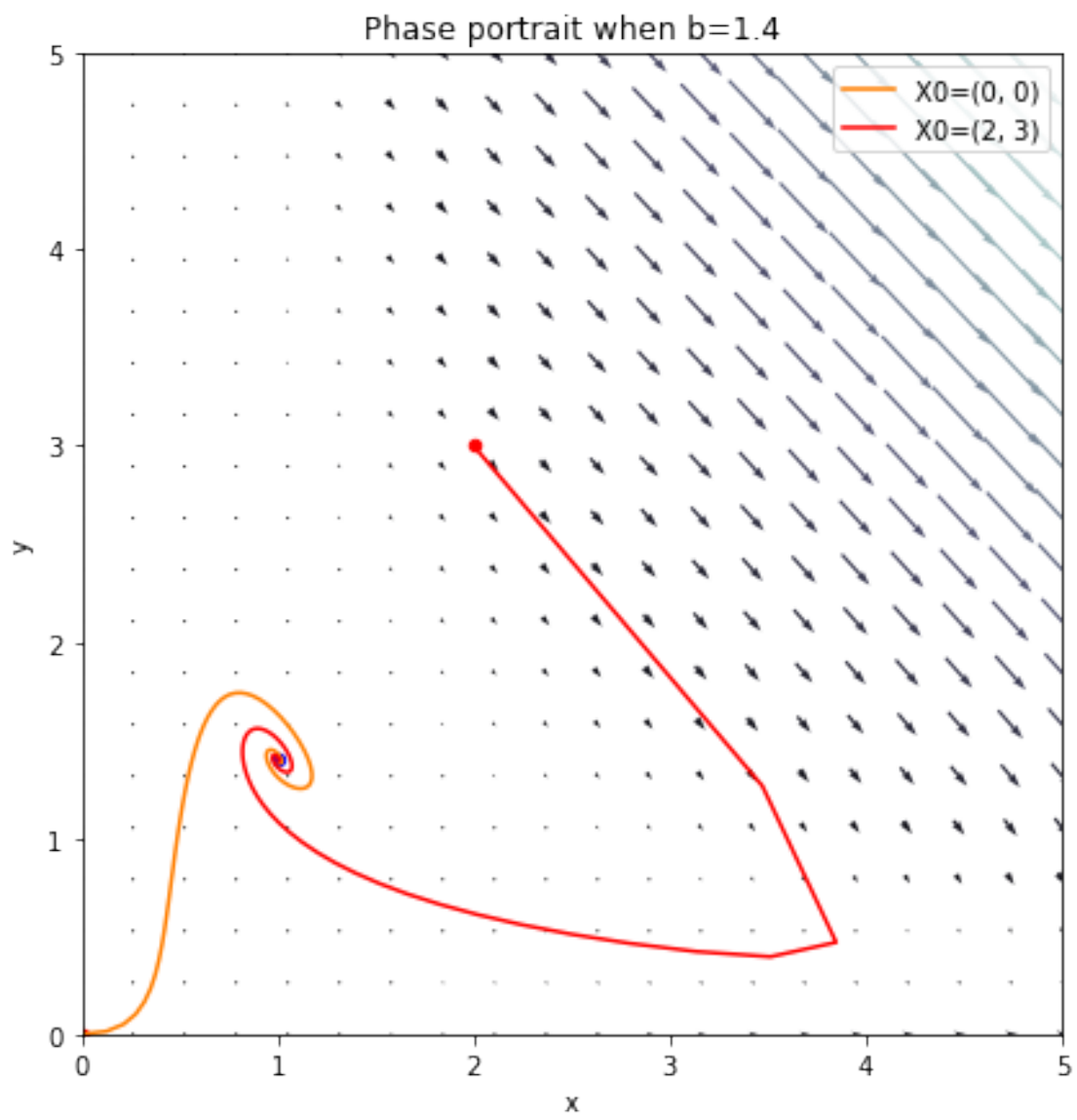
    ic_x = [ic[0] for ic in ics]
    ic_y = [ic[1] for ic in ics]
    ax.scatter(ic_x, ic_y, color='red', s=20)

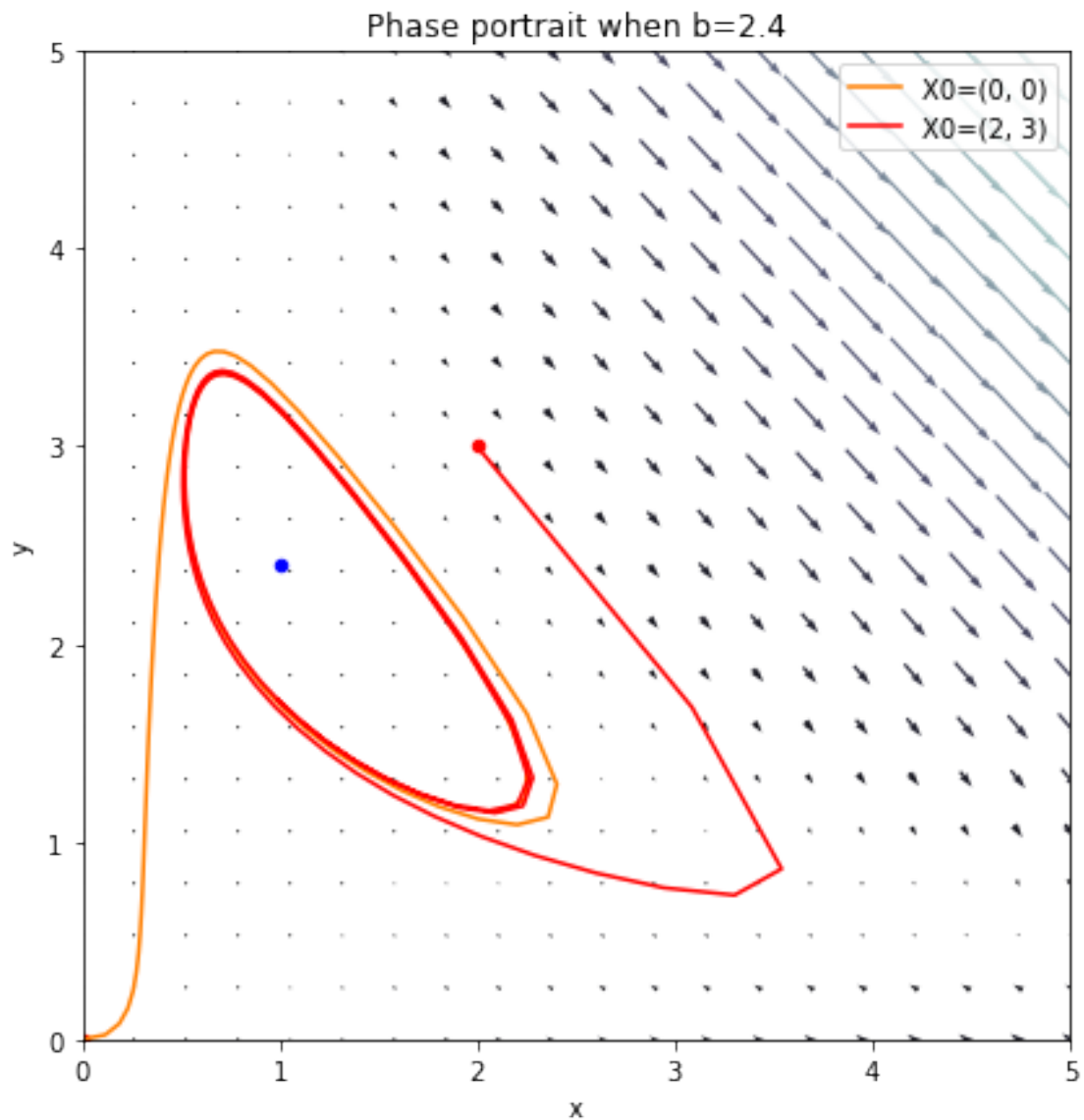
    ax.scatter(1, b, color='blue', s=20)

    plt.title('Phase portrait when b=' + str(b))
    plt.xlabel('x')
    plt.ylabel('y')
    plt.xlim(0,5)
    plt.ylim(0,5)
    plt.legend()
    plt.gca().set_aspect(1.0)

```

```
plt.show()
```





```
[27]: #
x = sym.Function('x')
y = sym.Function('y')
t = sym.Symbol("t")
a = sym.Symbol("a")
b = sym.Symbol("b")
u = sym.Function('u')
v = sym.Function('v')

eq1 = sym.Eq(x(t).diff(t), a - x(t) - b*x(t) + x(t)**2*y(t))
eq2 = sym.Eq(y(t).diff(t), b*x(t) - x(t)**2*y(t))
[eq1, eq2]
```



```

FG = sym.Matrix([eq1.rhs, eq2.rhs])
matJ = FG.jacobian([x(t), y(t)])
lin_mat = matJ.subs({x(t):a, y(t):(b/a)})
lin_mat.eigenvects()[0][2][0], lin_mat.eigenvects()[1][2][0]
evals = list(lin_mat.eigenvals().keys())

print('Eigenvalues for a = 1 and b = 1.8')
display_latex(evals[0].subs({a:1,b:1.8}))
display_latex(evals[1].subs({a:1,b:1.8}))

```

Eigenvalues for a = 1 and b = 1.8

$-0.1 - 0.99498743710662i$

$-0.1 + 0.99498743710662i$

```

[28]: print('Eigenvalues for a = 1 and b = 2.2')
display_latex(evals[0].subs({a:1,b:2.2}))
display_latex(evals[1].subs({a:1,b:2.2}))

```

Eigenvalues for a = 1 and b = 2.2

$0.1 - 0.99498743710662i$

$0.1 + 0.99498743710662i$

Plots show that the trajectories for $b = 1.4$ gose towards the critial point, while those for $b = 2.4$ asymptote towards a limit cycle. The real parts of eigenvalues for $b = 1.4$ and $b = 2.4$ have the same magnitiude but opposite sign, but their complex parts are the same. When $b = 1.4$, the nature of the critial point is a stable spiral sink. When $b = 2.4$, the nature of the critial point is a unstable spiral source.

3.12 (D)

```

[46]: fig, axes = plt.subplots(1, 2, figsize=(20, 8))
# Define the equation
def dX_dt(X, t):
    x1, x2 = X
    return [1- x1 - 3*x1 + x1**2*x2, 3*x1 - x1**2*x2]
X0 = [2, 3]
t = np.linspace(0, 40, 1000)
Xsol = odeint(dX_dt, X0, t)

axes[0].plot(t, Xsol[:, 0])
axes[0].set_title("$x_1(t)$")
axes[0].set_xlabel("$t$")
axes[0].set_ylabel("$x_1$")
axes[0].set_xlim(0,40)
axes[0].grid(True)

```

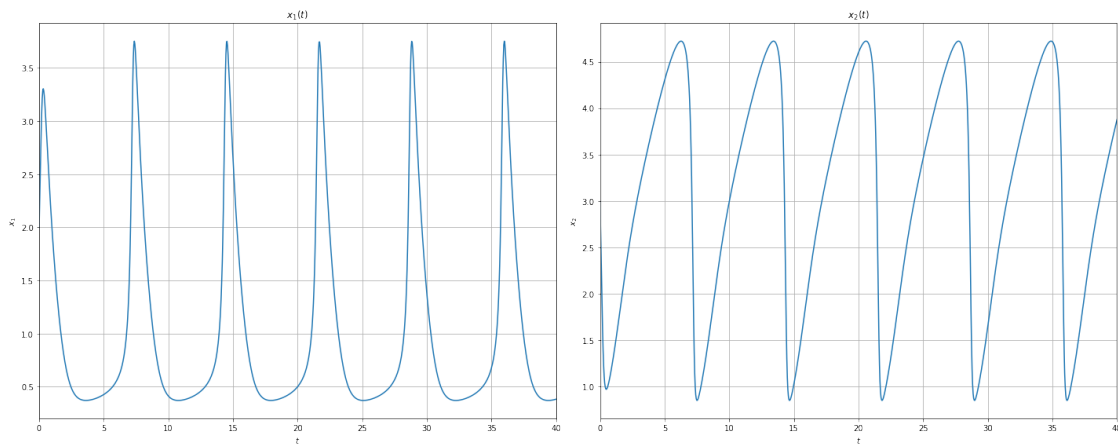
```

axes[1].plot(t, Xsol[:, 1])
axes[1].set_title("$x_2(t)$")
axes[1].set_xlabel("$t$")
axes[1].set_ylabel("$x_2$")
axes[1].set_xlim(0,40)
axes[1].grid(True)

fig.tight_layout()

plt.show()

```



[23]: *#We want to combine the 2 plots above into one graph.*

```

fig, axes = plt.subplots()

axes.plot(t, Xsol[:, 0],label='$x_1(t)$',)
axes.plot(t, Xsol[:, 1],label='$x_2(t)$')
axes.set(xlabel='t',
         title='Behaviour of x(t) and y(t) for a=1 and b=3')
axes.legend()
axes.grid(True)
fig.tight_layout()

plt.show()

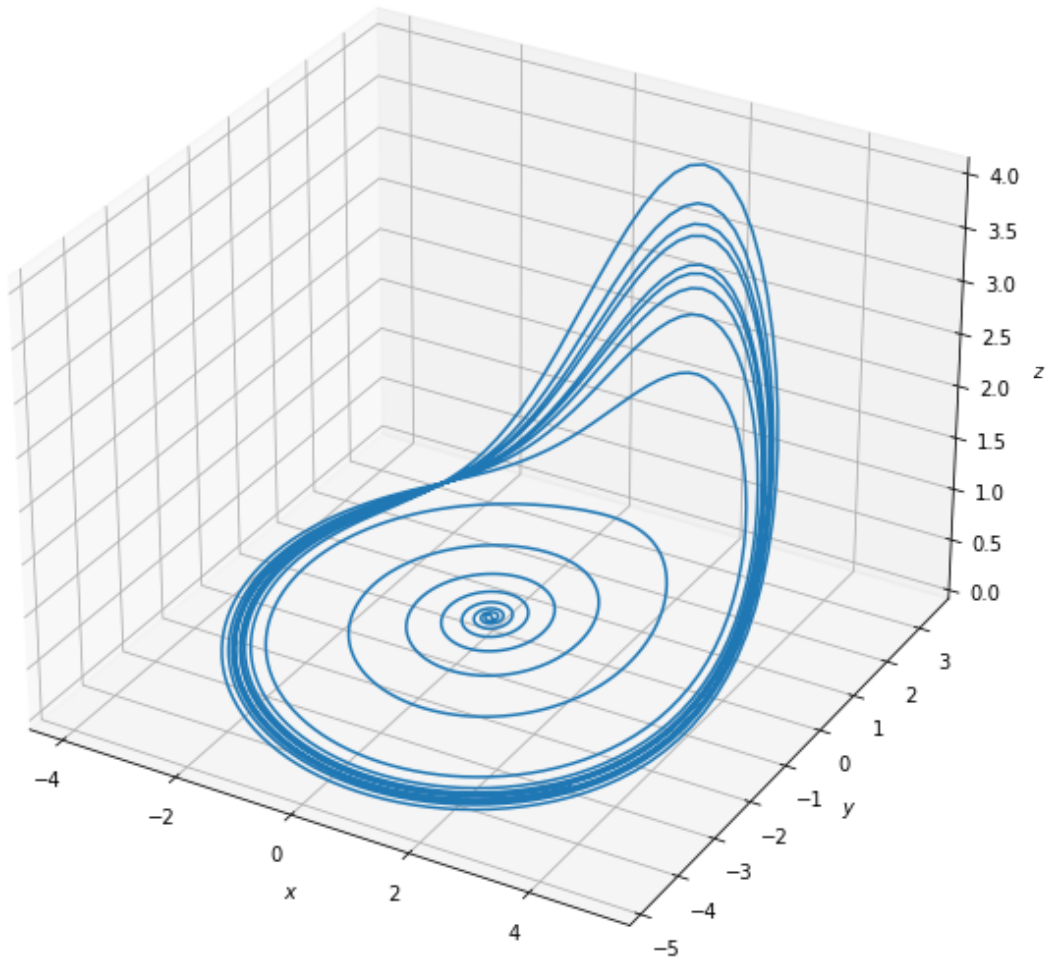
```

3.18 Question 4

3.19 (A)

```
[48]: def Rossler_dX_dt(X, t):  
        x, y, z = X  
        return [-y-z, x+(1/5)*y, (1/5) + (x-(5/2))*z]  
  
X0 = [0, 0, 0]  
t = np.linspace(0, 100, 2000)  
Xsolu = odeint(Rossler_dX_dt, X0, t)  
from mpl_toolkits.mplot3d import Axes3D  
  
# Set up figure & 3D axis  
fig = plt.figure(figsize=(10, 10))  
ax = fig.add_subplot(projection='3d')  
  
# Plot the solution  
plt.plot(Xsolu[:, 0], Xsolu[:, 1], Xsolu[:, 2])  
  
ax.set_xlabel('$x$')  
ax.set_ylabel('$y$')  
ax.set_zlabel('$z$')
```

```
plt.show()
```



3.20 (B)

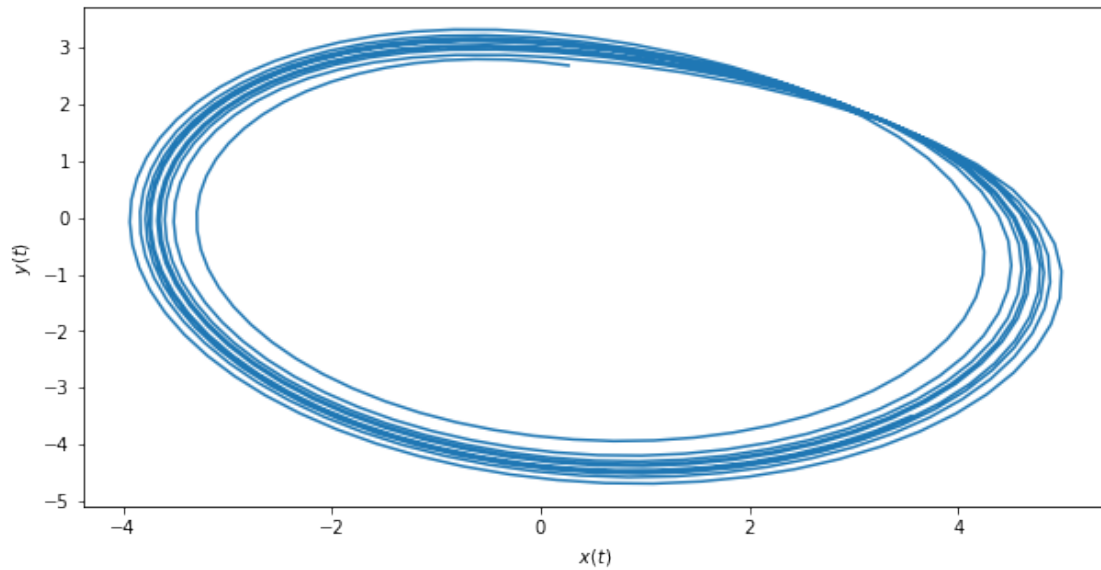
```
[49]: fig = plt.figure(figsize=(10, 5))
      ax = fig.add_subplot()

      t = np.linspace(0, 100, 1000)
      Xsol = odeint(Rossler_dX_dt, X0, t)
      plt.plot(Xsol[:, 0][440:], Xsol[:, 1][440:])

      ax.set_xlabel('$x(t)$')
```

```
ax.set_ylabel('$y(t)$')

plt.show()
```



```
[55]: fig, axes = plt.subplots(1, 2, figsize=(20, 8))
X0 = [0,0,0]
t = np.linspace(0, 100, 1000)
Xsol = odeint(Rossler_dX_dt, X0, t)

axes[0].plot(t[500:], Xsol[500:, 0])
axes[0].set_title("$x_1(t)$")
axes[0].set_xlabel("$t$")
axes[0].set_ylabel("$x_1$")

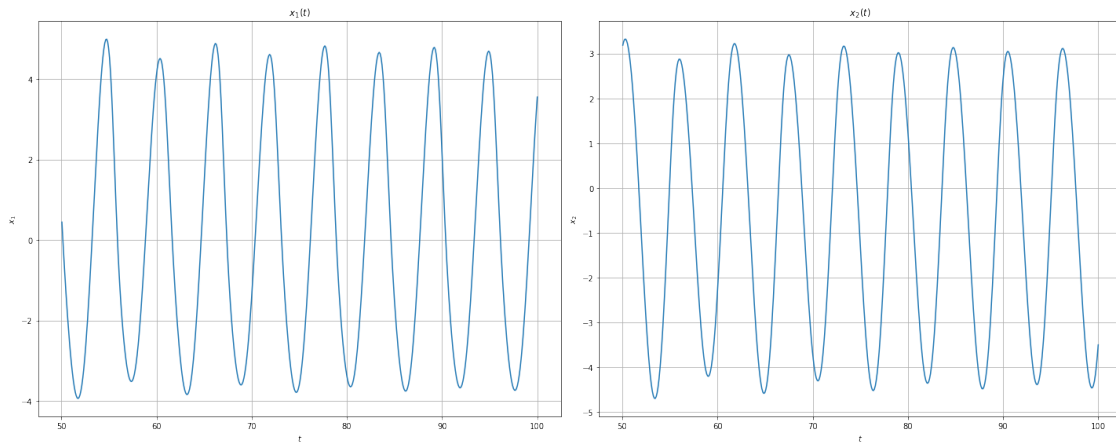
axes[0].grid(True)

axes[1].plot(t[500:], Xsol[500:, 1])
axes[1].set_title("$x_2(t)$")
axes[1].set_xlabel("$t$")
axes[1].set_ylabel("$x_2$")

axes[1].grid(True)

fig.tight_layout()

plt.show()
```



3.21 (C)

```
[44]: def Rossler_dX_dt_3(X, t):
        x, y, z = X
        return [-y-z, x+(1/5)*y, (1/5) + (x-3)*z]

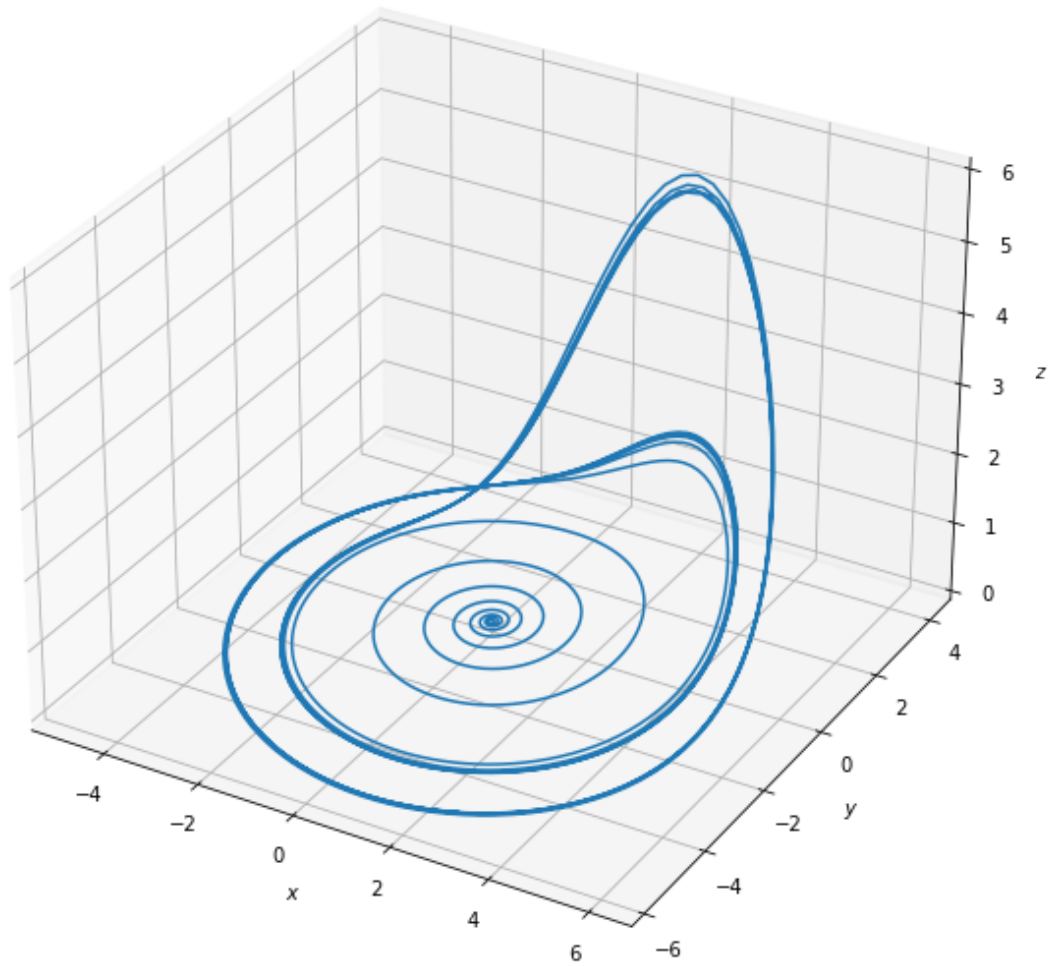
X0 = [0, 0, 0]
t = np.linspace(0, 100, 2000)
Xsol_3 = odeint(Rossler_dX_dt_3, X0, t)
from mpl_toolkits.mplot3d import Axes3D

# Set up figure & 3D axis
fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d')

# Plot the solution
plt.plot(Xsol_3[:, 0], Xsol_3[:, 1], Xsol_3[:, 2])

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
ax.set_zlabel('$z$')

plt.show()
```

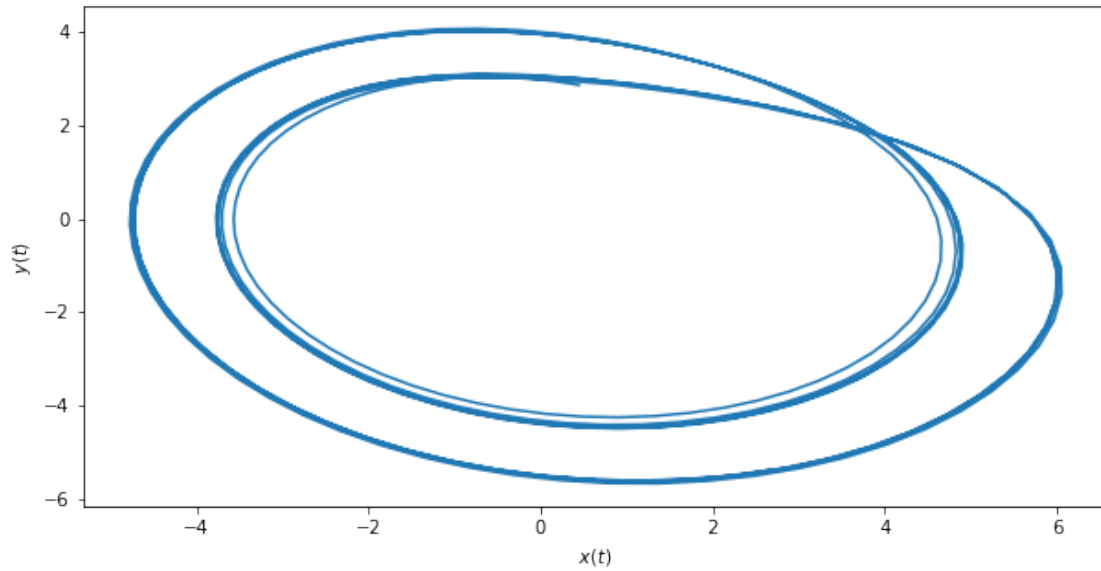


```
[45]: fig = plt.figure(figsize=(10, 5))
ax = fig.add_subplot()

t = np.linspace(0, 100, 1000)
Xsol = odeint(Rossler_dX_dt_3, X0, t)
plt.plot(Xsol[:, 0][440:], (Xsol[:, 1][440:]))

ax.set_xlabel('$x(t)$')
ax.set_ylabel('$y(t)$')

plt.show()
```



3.22 When I replace the coefficient $5/2$ appearing in the third equation with 3 , it appears two periodic orbits which are 2 winding orbits around the central equilibrium.

[]: